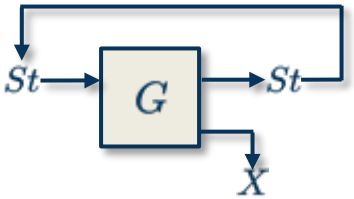
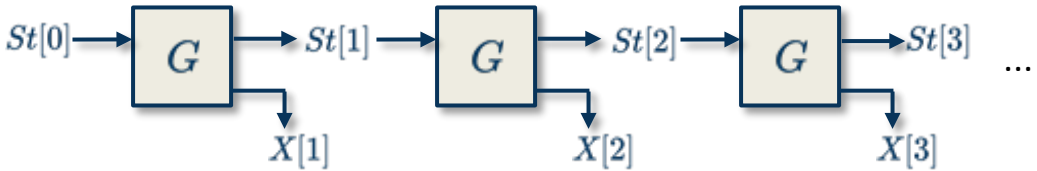
PSEUDORANDOM GENERATORS, STREAM CIPHERS

Stateful Generators

Initially, St is a random seed



Operation:



$X[1]X[2]X[3]\ldots$ is the output sequence and should be “pseudorandom”.

Notation

$$(X[1] \dots X[m], St) \leftarrow G(St, m)$$

means we

- Run G with starting state St for m steps
- Let $X[1] \dots X[m]$ be the output blocks produced
- Let St be the update state



Usage for Pseudorandom Bit Generation

- G is initialized with a random seed and its outputs are then used coins for any purpose needing randomness, including:
  - Keys
  - IVs for blockcipher-based encryption modes
  - ...

Use for Encryption

Alice maintains a state $\mathrm{St}_{\mathcal{A}}$ and Bob maintains a state $\mathrm{St}_{\mathcal{B}}$.

Initially: $\mathrm{St}_{\mathcal{A}} = \mathrm{St}_{\mathcal{B}}$ is a random seed.

$\mathcal{E}(M[1] \dots M[m])$:
$(X[1] \dots X[m], \mathrm{St}_{\mathcal{A}}) \leftarrow G(\mathrm{St}_{\mathcal{A}}, m)$
for $i = 1, \dots, m$ do
$C[i] \leftarrow X[i] \oplus M[i]$

$\mathcal{D}(C[1] \dots C[m])$:
$(X[1] \dots X[m], \mathrm{St}_{\mathcal{B}}) \leftarrow G(\mathrm{St}_{\mathcal{B}}, m)$
for $i = 1, \dots, m$ do
$M[i] \leftarrow X[i] \oplus C[i]$

Note that the states must be synchronized.

Methods

- Linear Congruential Generators (LCGs)
- Linear Feedback Shift Registers (LFSRs)

These have

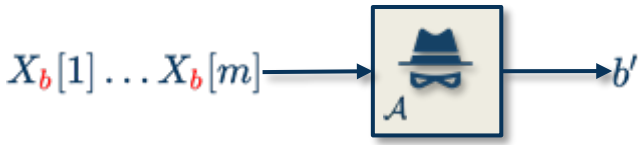
- Good statistical properties: #1's ≈ #0's. . .
- But are predictable: Given some outputs can infer future ones

Cryptographic constructs

- RC4
- SEAL

Theoretical Security Requirement

- INDR: Indistinguishability from random
  - Pick a random seed St and let $(X_1[1] \dots X_1[m], St) \leftarrow G(St, m)$
  - Pick $X_0[1] \dots X_0[m]$ at random
  - Pick a challenge bit b at random



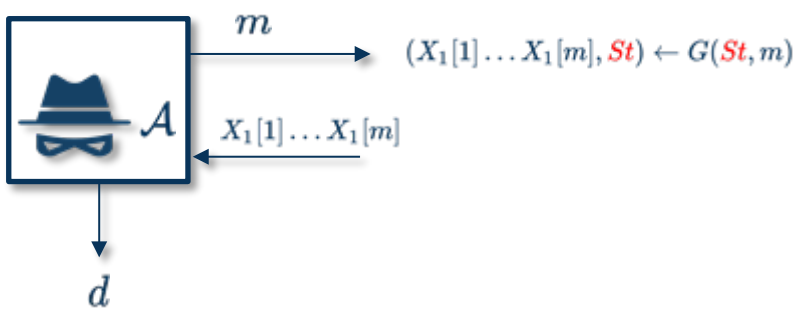
$\mathcal{A}$ wins if it guesses b

Formally

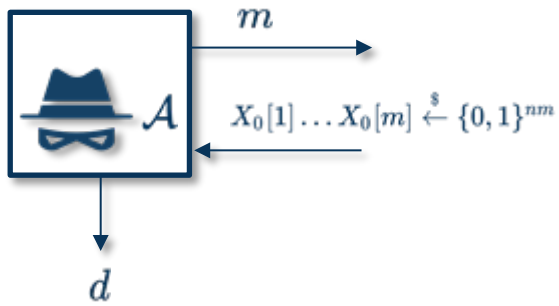
- Let G be a stateful generator with seed length s and output-block length n

Game INDR_G-1 ("real")

$St \xleftarrow{\$} \{0,1\}^s$



Game INDR_G-0 ("random")



The INDR advantage of $\mathcal{A}$ (denoted as $\mathrm{Adv}^{\mathrm{indr}}(\mathcal{A})$) is

$$\Pr[\mathcal{A} \Rightarrow 1 \text{ in Game } \mathsf{INDR}_G\text{-}1] - \Pr[\mathcal{A} \Rightarrow 1 \text{ in Game } \mathsf{INDR}_G\text{-}0]$$

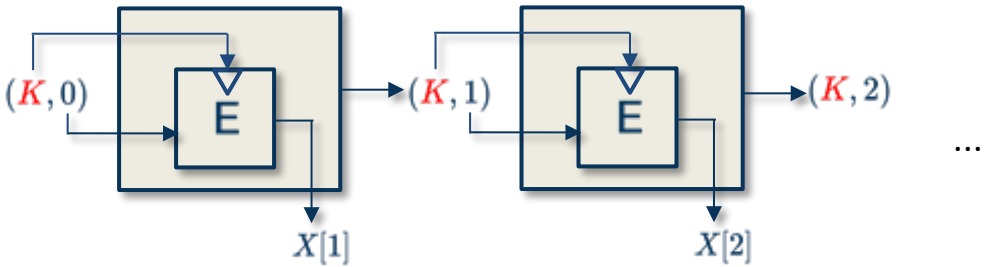
A stateful generator is indistinguishable from random (INDR-secure) if for any adversary $\mathcal{A}$ with "reasonable" resources its INDR advantage is "small" (close to 0).

PRGs from Blockciphers

- Let $\mathsf{E} : \{0,1\}^k \times \{0,1\}^n \to \{0,1\}^n$ be a blockcipher, and define:

$$\begin{array}{l}\textbf{Algorithm } G(St)\\ \quad (K,i) \leftarrow St \quad \text{// Parse } St \text{ as } (K,i)\\ \quad X \leftarrow \mathsf{E}_K(i+1); \textbf{return } (X,(K,i+1))\end{array}$$

  - State has the form where K is a key for E and i is an n-bit integer ($0 \le i < 2^n$).
  - Initial state is $(K,0)$ where $K \xleftarrow{\$} \{0,1\}^k$.



Security of CTR-PRG

Claim: If E is a secure PRF, then G is an INDR secure PRG.

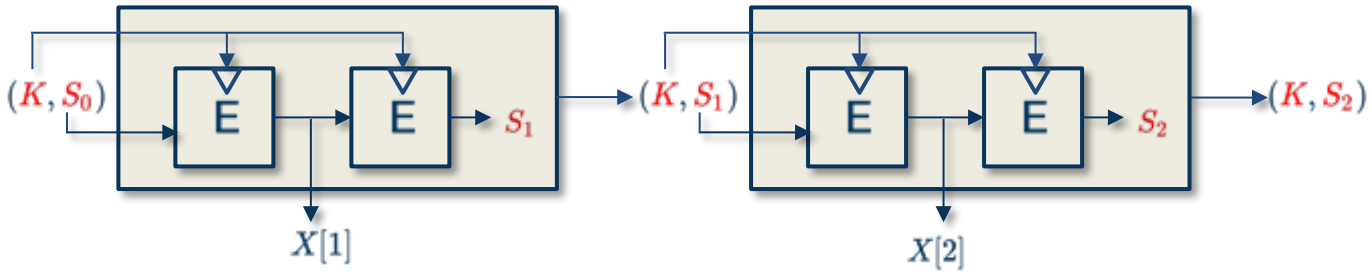
Similarly, other modes of operation of blockciphers also give rise to PRGs.

ANSI X9.17 PRG

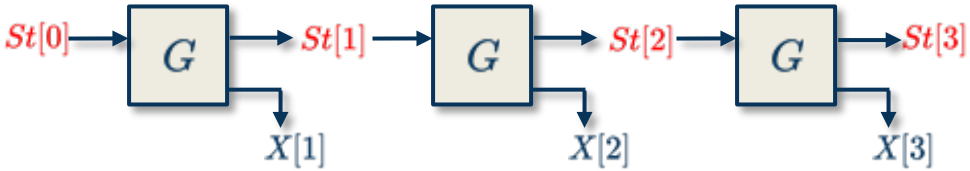
- Let $\mathsf{E} : \{0,1\}^k \times \{0,1\}^n \to \{0,1\}^n$ be a blockcipher, and define:

Algorithm $G(St)$
  $(K, S) \leftarrow St$
  $X \leftarrow \mathsf{E}_K(S); S \leftarrow \mathsf{E}_K(X);$
  return $(X, (K, S))$

  - State has the form (K, S) where K is a key for E and $S \in \{0,1\}^n$
  - initial state had both K and S chosen at random.



Forward Security

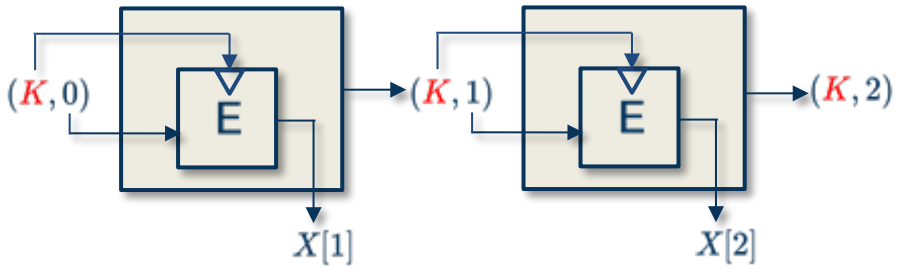


- Suppose adversary obtains $St[2]$. Then
  - It can compute $X[3]X[4]$
  - But can it compute $X[1]X[2]$?

    Forward security requires that the answer to the latter question be “NO”.

    Important in the face of exposure due to malware and system compromise.

Forward Security Failures



If adversary gets $(K, 2)$ then it can compute

$$X[1] = \mathsf{E}_K(0); X[2] = \mathsf{E}_K(1).$$

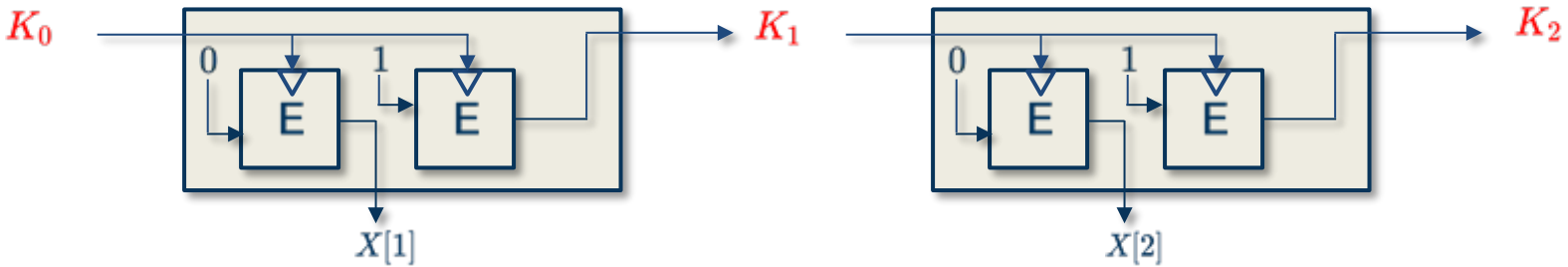
Similar failures for ANSI X9.17

Forward-Secure PRG

- Let $\mathsf{E} : \{0,1\}^k \times \{0,1\}^n \to \{0,1\}^n$ be a blockcipher, and define:

$$\begin{array}{l}\textbf{Algorithm } G(K) \\ \quad X \leftarrow \mathsf{E}_K(0); K \leftarrow \mathsf{E}_K(1); \\ \quad \textbf{return } (X, K)\end{array}$$

  - State is a key K for E
  - Initial state is a random $K \xleftarrow{\$} \{0,1\}^n$



The Full Process

- In practice, random number generation (RNG) involves
  - Seeding
  - PRG
- Failures in RNG are common. They arise from failures in either component. In principle, we know how to design good PRGs. But seeding remains a problem
- Typical methods:
  - Maintain an “entropy pool” based on system events, user keystrokes, …
  - Mix in more entropy as needed